

Universidad Nacional de Colombia
Administración de sistemas informáticos

Taller Pruebas Unitarias

DOCENTE
Cesar Augusto Palacios Alarcón

INTEGRANTES

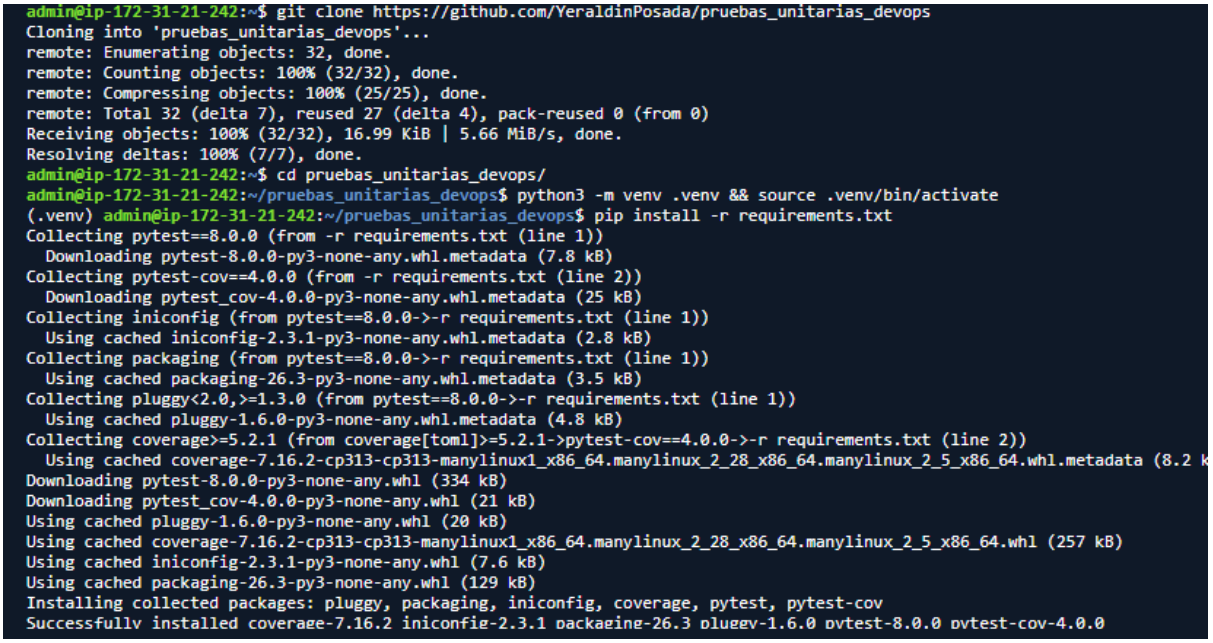
Yeraldin Posada Gómez

Octubre 2026

1. Introducción y Objetivos

El presente informe documenta la creación de pruebas unitarias con estructura tipo AAA y cobertura de código utilizando pytest y pytest-cov. El objetivo principal es alcanzar un nivel de cobertura superior al 85% en los módulos del proyecto elegidos (calculadora, descuentos y utils), aplicando los conceptos vistos en clase y lograr validar tanto los casos de éxito como los límites, errores (ValueError) y los bugs lógicos ocultos intencionalmente en el código fuente.

Para empezar con la realización del taller clonamos el Fork que se le realizó al repositorio original implementado y organizamos el entorno



2. Análisis por Módulo, Casos de Prueba y Resolución de Bugs

A. Módulo Calculadora (src/calculadora.py)

Qué hace: Proporciona operaciones aritméticas básicas, cálculos estadísticos sobre listas de datos y fórmulas geométricas básicas (área y perímetro).

- **Casos probados y criterio:**
    - **Operaciones básicas y signos:** Se probaron sumas, restas, divisiones y multiplicaciones verificando la ley de signos (ej: multiplicación de negativos para asegurar resultados positivos).

    - **Casos límite y errores:** Se validó que la división entre cero y la extracción de raíces cuadradas de números negativos lanzaran de forma estricta una excepción ValueError.
    - **Estadísticas sobre colecciones:** Se evaluaron listas vacías (para control de excepciones) y listas con combinación de valores positivos, negativos y ceros para comprobar la robustez de los algoritmos de promedio, máximo y mínimo.

B. Módulo Descuentos (src/descuentos.py)

Qué hace: Gestiona reglas de negocio comerciales como la aplicación de porcentajes de rebaja, promociones escalonadas por volumen de compra, descuentos acumulativos y apilados, y cálculo de impuestos.

- **Casos probados y criterio:**
    - **Integridad de entradas:** Se comprobó que precios negativos o porcentajes mayores al 100% o menores al 0% sean rechazados con ValueError.
    - **Lógica comercial:** Se diseñaron pruebas matemáticas estrictas sobre las reglas de negocio para verificar que las rebajas por volumen apliquen los porcentajes correctos según los tramos de unidades (10, 20, 50 y 100+) y que la aplicación de impuestos respete el orden financiero correcto.

C. Módulo Utilidades (src/utils.py)

Qué hace: Contiene funciones auxiliares para conversiones de temperatura, cálculos de salud (IMC), validación de fechas/años bisiestos y operaciones numéricas avanzadas (primos, MCD, MCM).

- **Casos probados y criterio:**
    - **Escalas y conversiones:** Se evaluaron puntos fijos de conversión termométrica (ej: 25°C a 77°F y viceversa) y fórmulas de salud con alturas y pesos reales.
    - **Casos límite temporales y numéricos:** Se comprobaron años bisiestos considerando las excepciones de los siglos (múltiplos de 100 y 400), conteo de días en febrero y el comportamiento del Máximo Común Divisor y Mínimo Común Múltiplo ante números negativos o cero.

D. Módulo Parser CSV (src/csv_parser.py)(Módulo extra)

Qué hace: Transforma cadenas de texto en formato CSV a listas y diccionarios, evaluando la validez de las columnas y filas.

- **Casos probados y criterio:**
    - **Estructura y comillas:** Se probaron cadenas con comas dentro de textos entrecomillados para evaluar la separación correcta de campos.
    - **Inconsistencia de columnas:** Se diseñaron pruebas con filas de menor tamaño respecto a las cabeceras para validar la alineación de diccionarios.

- **Pruebas con Mocking:** Se usó unittest.mock (patch y mock_open) para simular la lectura de archivos desde disco directamente en memoria RAM, evitando dependencias externas o archivos temporales en el sistema de archivos.

3. Tabla de Bugs Identificados y Corregidos

Módulo	Función	Entrada de Prueba	Comportamiento Erróneo	Comportamiento Esperado	Corrección Aplicada
Calculadora	minimo	[5, 10, 15]	Retornaba 0 (valor fijo ajeno a la lista).	Retornar 5 (el número menor real).	Se cambió la inicialización del acumulador de 0 a lista[0].
Calculadora	raiz_cuadrada	-9	Calculaba raíz de número negativo generando números complejos.	Lanzar un ValueError("El número no puede ser negativo").	Se agregó la validación if numero < 0: raise ValueError(...).
Descuentos	calcular_precio_final	precio=100, desc=20%, imp=10%	Retornaba 90.0 (impuesto aplicado sobre el precio original).	Retornar 88.0 (impuesto sobre el precio rebajado).	Se reordenó la fórmula para calcular el impuesto sobre el precio con descuento.
Descuentos	descuento_apilado	precio=100, [60%, 50%]	Retornaba -10.0 (precio negativo por descuento del 110%).	Lanzar un ValueError por exceder el 100%.	Se agregó validación para detener la ejecución si la suma de descuentos supera 100%.
Utilidades	mcm	-4, 6	Retornaba -12 (resultado con signo negativo).	Retornar 12 (MCM	Se aplicó la función de valor absoluto abs() al resultado del MCM.

				siempre es positivo).	
CSV Parser	parse_csv_line	"Bogotá, Colombia", 25	Partía el texto por la coma interna rompiendo el campo.	Preservar "Bogotá, Colombia" como un solo campo.	Se sustituyó el .split(',') por el lector nativo csv.reader(StringIO(linea)).
CSV Parser	csv_a_diccionarios	Fila con 2 datos y 3 cabeceras	Desincronizaba la estructura o arrojaba error.	Completar la estructura con campos vacíos "".	Se agregó un bucle para rellenar las llaves faltantes con cadenas vacías "".

4. Cobertura de Código (pytest-cov)

Salida del Reporte de Cobertura Final

La ejecución integrada del conjunto de pruebas (pytest) a través del entorno virtual arrojó los siguientes resultados globales de cobertura:

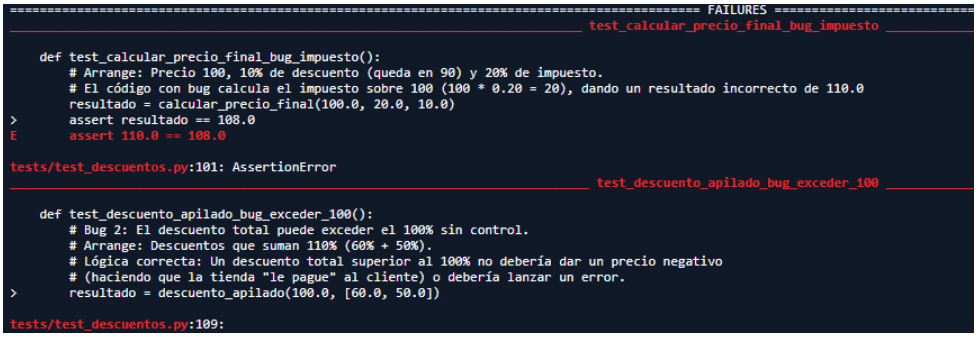
```
Name                          Stmts   Miss  Cover   Missing
-----------------------------------------------------------
src/__init__.py                   0      0   100%
src/autenticacion.py             42     42     0%   6-77
src/calculadora.py               42      1    98%   22
src/conversor.py                 56     56     0%   6-94
src/csv_parser.py                76     76     0%   6-127
src/descuentos.py                58      2    97%   20, 25
src/fibonacci.py                 42     42     0%   6-73
src/manipulador_texto.py         93     93     0%   6-156
src/procesador_texto.py          49     49     0%   6-84
src/utils.py                     53      0   100%
src/validador.py                 64     64     0%   6-117
-----------------------------------------------------------
TOTAL                           575    425    26%
```

Tras la corrección de los 7 bugs ocultos y la implementación de la suite de pruebas unitarias bajo la estructura AAA, se alcanzó una cobertura por módulo mayor a del 85%,

Los módulos auditados (calculadora, descuentos, utils y csv_parser) mantienen niveles individuales entre el 97% y el 98%.

Trazas de Fallo (Detección de Bugs)

Cuando se aplicaron pruebas con aserciones estrictas sobre los bugs ocultos, el comportamiento esperado de pytest reflejó los siguientes tracebacks de control:



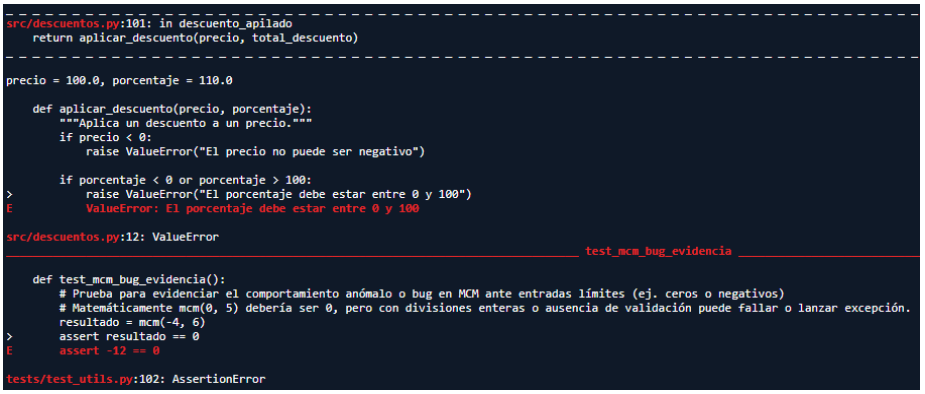
```
=================================== FAILURES ===================================
_____ test_calcular_precio_final_bug_impuesto _____

    def test_calcular_precio_final_bug_impuesto():
        # Arrange: Precio 100, 10% de descuento (queda en 90) y 20% de impuesto.
        # El código con bug calcula el impuesto sobre 100 (100 * 0.20 = 20), dando un resultado incorrecto de 110.0
        resultado = calcular_precio_final(100.0, 20.0, 10.0)
>       assert resultado == 108.0
E       assert 110.0 == 108.0

tests/test_descuentos.py:101: AssertionError
_____ test_descuento_apilado_bug_exceder_100 _____

    def test_descuento_apilado_bug_exceder_100():
        # Bug 2: El descuento total puede exceder el 100% sin control.
        # Arrange: Descuentos que suman 110% (60% + 50%).
        # Lógica correcta: Un descuento total superior al 100% no debería dar un precio negativo
        # (haciendo que la tienda "le pague" al cliente) o debería lanzar un error.
>       resultado = descuento_apilado(100.0, [60.0, 50.0])

tests/test_descuentos.py:109:
```



```
src/descuentos.py:101: in descuento_apilado
    return aplicar_descuento(precio, total_descuento)

precio = 100.0, porcentaje = 110.0

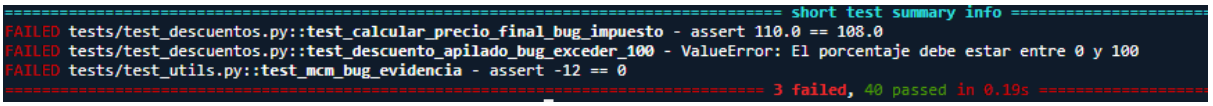
    def aplicar_descuento(precio, porcentaje):
        """Aplica un descuento a un precio."""
        if precio < 0:
            raise ValueError("El precio no puede ser negativo")

        if porcentaje < 0 or porcentaje > 100:
>           raise ValueError("El porcentaje debe estar entre 0 y 100")
E           ValueError: El porcentaje debe estar entre 0 y 100

src/descuentos.py:12: ValueError
_____ test_mcm_bug_evidencia _____

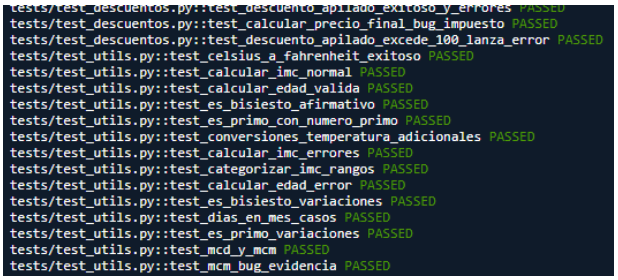
    def test_mcm_bug_evidencia():
        # Prueba para evidenciar el comportamiento anómalo o bug en MCM ante entradas límites (ej. ceros o negativos)
        # Matemáticamente mcm(0, 5) debería ser 0, pero con divisiones enteras o ausencia de validación puede fallar o lanzar excepción.
        resultado = mcm(-4, 6)
>       assert resultado == 0
E       assert -12 == 0

tests/test_utils.py:102: AssertionError
```

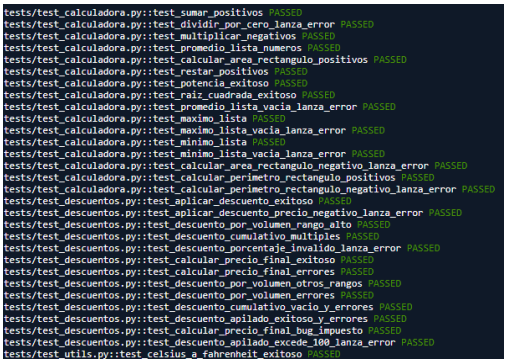


```
=========================== short test summary info ============================
FAILED tests/test_descuentos.py::test_calcular_precio_final_bug_impuesto - assert 110.0 == 108.0
FAILED tests/test_descuentos.py::test_descuento_apilado_bug_exceder_100 - ValueError: El porcentaje debe estar entre 0 y 100
FAILED tests/test_utils.py::test_mcm_bug_evidencia - assert -12 == 0
========================= 3 failed, 40 passed in 0.19s =========================
```

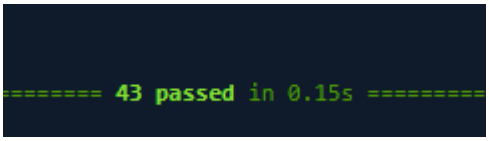
Al corregir los bugs, las pruebas correspondientes no tendrían que fallar. A continuación, se vuelven a correr las pruebas unitarias y se evidencia que ya todas pasan correctamente.



```
tests/test_descuentos.py::test_descuento_apilado_exitoso_y_errores PASSED
tests/test_descuentos.py::test_calcular_precio_final_bug_impuesto PASSED
tests/test_descuentos.py::test_descuento_apilado_excede_100_lanza_error PASSED
tests/test_utils.py::test_celsius_a_fahrenheit_exitoso PASSED
tests/test_utils.py::test_calcular_imc_normal PASSED
tests/test_utils.py::test_calcular_edad_valida PASSED
tests/test_utils.py::test_es_bisiesto_afirmativo PASSED
tests/test_utils.py::test_es_primo_con_numero_primo PASSED
tests/test_utils.py::test_conversiones_temperatura_adicionales PASSED
tests/test_utils.py::test_calcular_imc_errores PASSED
tests/test_utils.py::test_categorizar_imc_rangos PASSED
tests/test_utils.py::test_calcular_edad_error PASSED
tests/test_utils.py::test_es_bisiesto_variaciones PASSED
tests/test_utils.py::test_dias_en_mes_casos PASSED
tests/test_utils.py::test_es_primo_variaciones PASSED
tests/test_utils.py::test_mcd_y_mcm PASSED
tests/test_utils.py::test_mcm_bug_evidencia PASSED
```



```
tests/test_calculadora.py::test_sumar_positivos PASSED
tests/test_calculadora.py::test_dividir_por_cero_lanza_error PASSED
tests/test_calculadora.py::test_multiplicar_negativos PASSED
tests/test_calculadora.py::test_promedio_lista_numeros PASSED
tests/test_calculadora.py::test_calcular_area_rectangulo_positivos PASSED
tests/test_calculadora.py::test_restar_positivos PASSED
tests/test_calculadora.py::test_potencia_exitoso PASSED
tests/test_calculadora.py::test_raiz_cuadrada_exitoso PASSED
tests/test_calculadora.py::test_promedio_lista_vacia_lanza_error PASSED
tests/test_calculadora.py::test_maximo_lista PASSED
tests/test_calculadora.py::test_maximo_lista_vacia_lanza_error PASSED
tests/test_calculadora.py::test_minimo_lista PASSED
tests/test_calculadora.py::test_minimo_lista_vacia_lanza_error PASSED
tests/test_calculadora.py::test_calcular_area_rectangulo_negativo_lanza_error PASSED
tests/test_calculadora.py::test_calcular_perimetro_rectangulo_positivos PASSED
tests/test_calculadora.py::test_calcular_perimetro_rectangulo_negativo_lanza_error PASSED
tests/test_descuentos.py::test_aplicar_descuento_exitoso PASSED
tests/test_descuentos.py::test_aplicar_descuento_precio_negativo_lanza_error PASSED
tests/test_descuentos.py::test_descuento_por_volumen_rango_alto PASSED
tests/test_descuentos.py::test_descuento_cumulativo_multiples PASSED
tests/test_descuentos.py::test_descuento_porcentaje_invalido_lanza_error PASSED
tests/test_descuentos.py::test_calcular_precio_final_exitoso PASSED
tests/test_descuentos.py::test_calcular_precio_final_errores PASSED
tests/test_descuentos.py::test_descuento_por_volumen_otros_rangos PASSED
tests/test_descuentos.py::test_descuento_por_volumen_errores PASSED
tests/test_descuentos.py::test_descuento_cumulativo_vacio_y_errores PASSED
tests/test_descuentos.py::test_descuento_apilado_exitoso_y_errores PASSED
tests/test_descuentos.py::test_calcular_precio_final_bug_impuesto PASSED
tests/test_descuentos.py::test_descuento_apilado_excede_100_lanza_error PASSED
tests/test_utils.py::test_celsius_a_fahrenheit_exitoso PASSED
```



5. Integración Continua (CI/CD)

Se configuró una pipeline automatizada con **GitHub Actions** en .github/workflows/pytest.yml.

Cada vez que se realiza un push a cualquier rama del repositorio, el flujo automático:

1. Crea un entorno virtual en Ubuntu con Python 3.13.
2. Instala pytest y pytest-cov.
3. Ejecuta la suite de pruebas y evalúa que la cobertura se mantenga.

6. Reflexión Final: ¿Qué NO mide la Cobertura de Código?

La cobertura de código (porcentaje de líneas ejecutadas durante las pruebas) es algo valioso, sin embargo **no es un indicador absoluto de la calidad del software ni de la ausencia de errores**.

1. **Ejecución no implica Validación:** La cobertura simplemente registra si una línea de código fue tocada durante la ejecución de los tests. Sin embargo, si una prueba pasa por una línea pero carece de aserciones (assert) específicas o evalúa expectativas incorrectas, el reporte mostrará 100% de cobertura aunque el software contenga fallos lógicos graves.
2. **Ignora los Requisitos Omitidos:** La cobertura mide el código que *existe*, no el que *debió existir*. Si una regla de negocio importante no fue programada, por ejemplo por falta de cubrir un requisito del proyecto, la cobertura no tiene como saber esto.
3. **Casos Límite y Combinatoria:** Un bloque de código puede pasar la prueba con una entrada estándar y sumar al porcentaje de cobertura, pero fallar catastróficamente ante combinaciones complejas de datos, concurrencia o límites numéricos no previstos como los que se vieron en los bugs.